DDL Commands with Constraints in DBMS (MySQL)

DDL (Data Definition Language) commands are used to create, modify, and delete database structures such as tables.

Constraints are rules applied to table columns to ensure the accuracy, consistency, and integrity of the data stored in a database.

The six important SQL constraints are:

Constraint	Purpose
PRIMARY KEY	Uniquely identifies each record in a table. Cannot contain NULL.
FOREIGN KEY	Establishes a relationship between two tables and maintains referential integrity.
UNIQUE	Prevents duplicate values in a column.
CHECK	Ensures values satisfy a specified condition.
DEFAULT	Automatically assigns a predefined value when no value is provided.
NOT NULL	Prevents NULL values in a column.

1. PRIMARY KEY Constraint

A PRIMARY KEY uniquely identifies each row in a table.

Example: Create a Students table

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50),
    Age INT
);
```

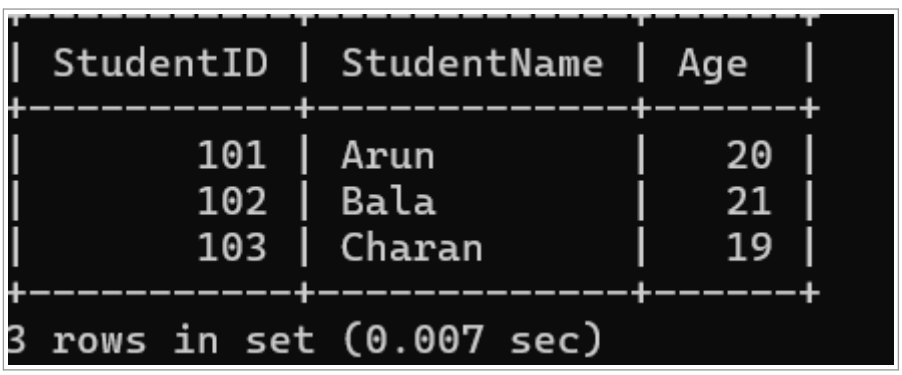
Insert records:

```
INSERT INTO Students VALUES (101, 'Arun', 20);
INSERT INTO Students VALUES (102, 'Bala', 21);
INSERT INTO Students VALUES (103, 'Charan', 19);
```

Result:

StudentID (PK)	StudentName	Age
101	Arun	20
102	Bala	21
103	Charan	19

*Output: SELECT * FROM Students; (all three records inserted)*



Violation:

```
INSERT INTO Students VALUES (101, 'David', 22);
```

Error: Duplicate entry '101' for PRIMARY KEY.

StudentID 101 already exists. Therefore, another record cannot use the same primary key value.

Output: PRIMARY KEY violation (duplicate StudentID 101)

```
mysql> INSERT INTO Students VALUES (101, 'David', 22);
ERROR 1062 (23000): Duplicate entry '101' for key 'students.PRIMARY'
```

2. FOREIGN KEY Constraint

A FOREIGN KEY establishes a relationship between two tables.

Consider two tables:

Departments table (Parent Table)

DepartmentID (PK)	DepartmentName
10	CSE
20	ECE
30	IT

Employees table (Child Table)

EmployeeID (PK)	EmployeeName	DepartmentID (FK)
101	Ravi	10
102	Priya	20
103	Kumar	10

Create both tables:

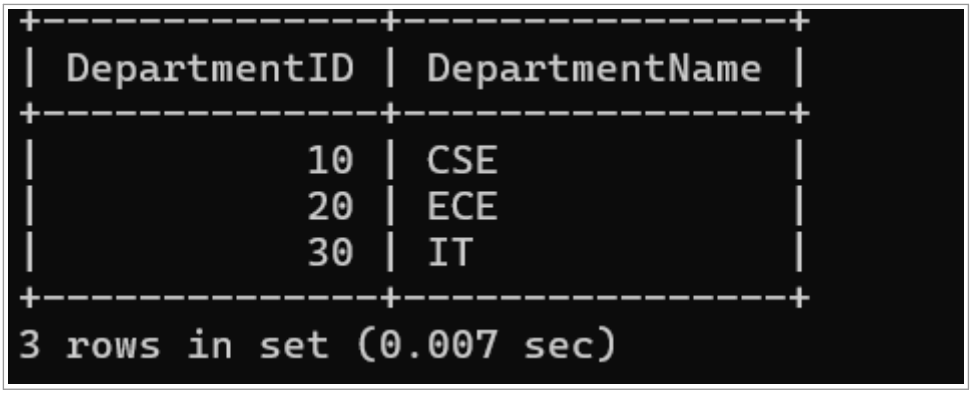
```
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50)
);

CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50),
    DepartmentID INT,
    FOREIGN KEY (DepartmentID)
        REFERENCES Departments(DepartmentID)
);
```

Insert department records first:

```
INSERT INTO Departments VALUES
(10, 'CSE'),
(20, 'ECE'),
(30, 'IT');
```

*Output: SELECT * FROM Departments; (parent table)*

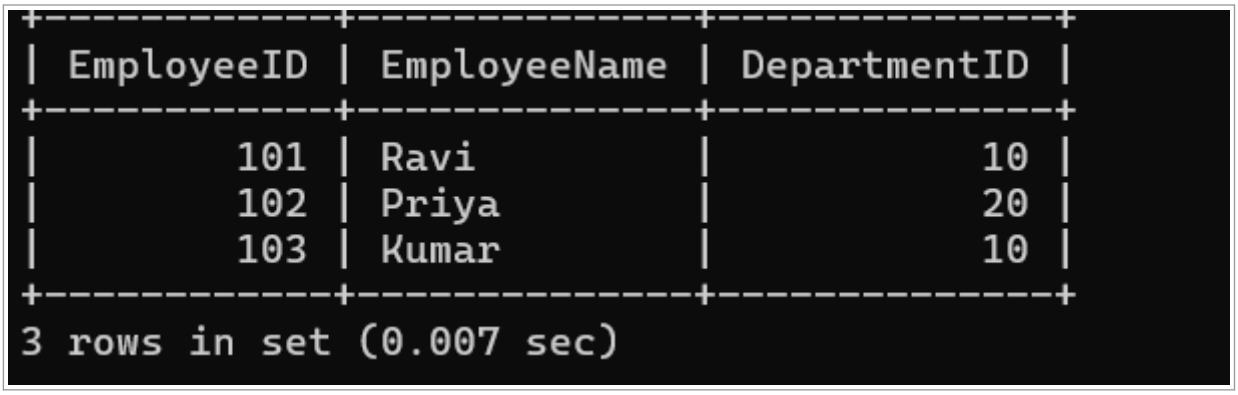


```
+--------------+----------------+
| DepartmentID | DepartmentName |
+--------------+----------------+
|           10 | CSE            |
|           20 | ECE            |
|           30 | IT             |
+--------------+----------------+
3 rows in set (0.007 sec)
```

Insert employee records:

```
INSERT INTO Employees VALUES
(101, 'Ravi', 10),
(102, 'Priya', 20),
(103, 'Kumar', 10);
```

*Output: SELECT * FROM Employees; (child table)*



```
+------------+--------------+--------------+
| EmployeeID | EmployeeName | DepartmentID |
+------------+--------------+--------------+
|        101 | Ravi         |           10 |
|        102 | Priya        |           20 |
|        103 | Kumar        |           10 |
+------------+--------------+--------------+
3 rows in set (0.007 sec)
```

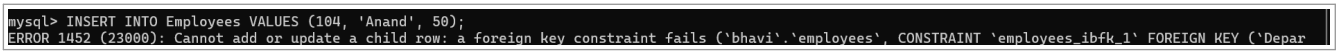
Violation:

```
INSERT INTO Employees VALUES (104, 'Anand', 50);
```

Error: Foreign key constraint fails.

DepartmentID 50 does not exist in the Departments table.

Output: FOREIGN KEY violation (Error 1452, DepartmentID 50 not in parent table)



```
mysql> INSERT INTO Employees VALUES (104, 'Anand', 50);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`bhavi`.`employees`, CONSTRAINT `employees_ibfk_1` FOREIGN KEY (`Depar
```

3. UNIQUE Constraint

The UNIQUE constraint ensures that duplicate non-NULL values are not inserted into a column.

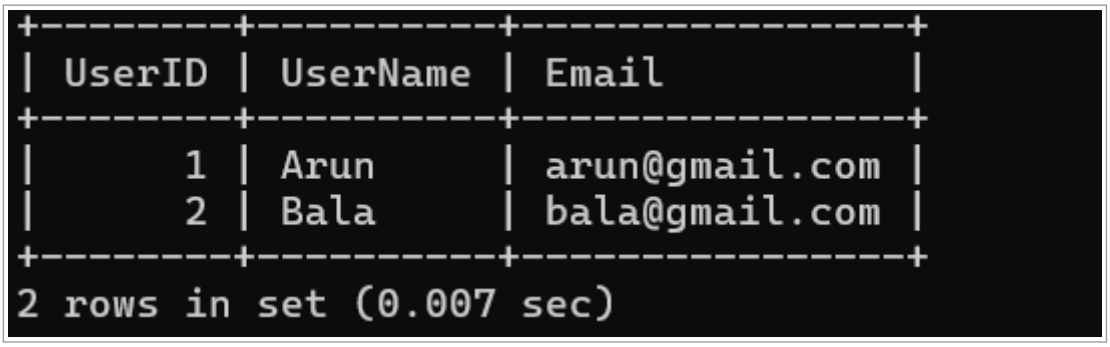
```
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    UserName VARCHAR(50),
    Email VARCHAR(100) UNIQUE
);
```

Insert records:

```
INSERT INTO Users VALUES
(1, 'Arun', 'arun@gmail.com'),
(2, 'Bala', 'bala@gmail.com');
```

UserID	UserName	Email (UNIQUE)
1	Arun	arun@gmail.com
2	Bala	bala@gmail.com

*Output: SELECT * FROM Users;*

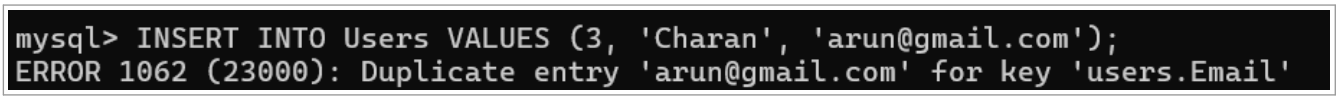


Violation:

```
INSERT INTO Users VALUES
(3, 'Charan', 'arun@gmail.com');
```

Error: Duplicate entry 'arun@gmail.com'.

Output: UNIQUE violation (duplicate Email)



4. CHECK Constraint

The CHECK constraint ensures that data satisfies a particular condition.

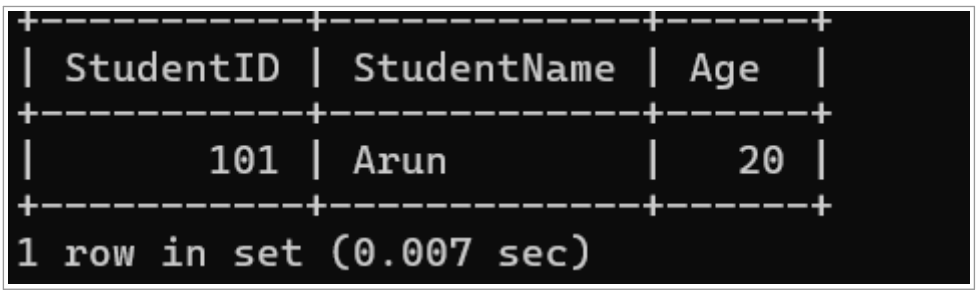
Example: Students' ages must be at least 18.

```
CREATE TABLE CollegeStudents (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50),
    Age INT CHECK (Age >= 18)
);
```

Valid insertion:

```
INSERT INTO CollegeStudents
VALUES (101, 'Arun', 20);
```

*Output: SELECT * FROM CollegeStudents; (only the valid record is stored)*



Invalid insertion:

```
INSERT INTO CollegeStudents
VALUES (102, 'Bala', 16);
```

Error: CHECK constraint violated because Age = 16.

Output: CHECK violation (Age = 16)



Another example:

```
CREATE TABLE ExamResults (
    StudentID INT PRIMARY KEY,
    Marks INT CHECK (Marks BETWEEN 0 AND 100)
);
```

This allows marks between 0 and 100.

```
INSERT INTO ExamResults VALUES (101, 85);
```

Valid.

```
INSERT INTO ExamResults VALUES (102, 120);
```

Invalid because 120 is greater than 100.

Note: CHECK does not automatically prevent NULL. Add NOT NULL when a value is mandatory.

5. DEFAULT Constraint

The DEFAULT constraint automatically inserts a predefined value when a column value is omitted.

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Status VARCHAR(20) DEFAULT 'Pending'
);
```

Insert records:

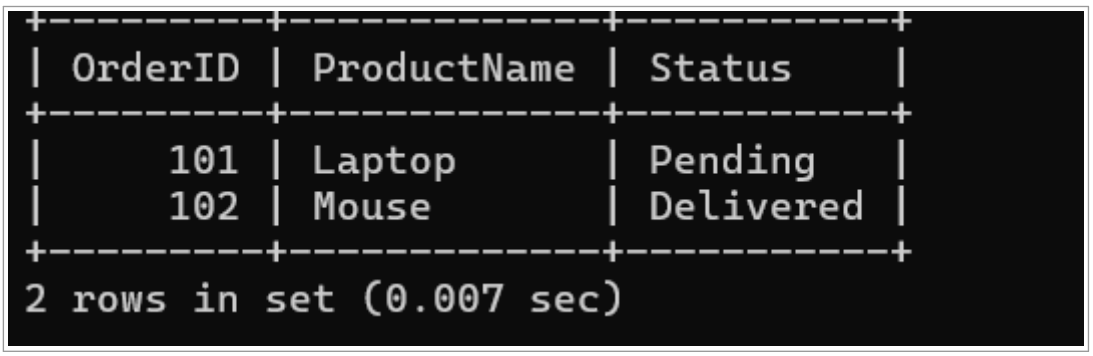
```
INSERT INTO Orders (OrderID, ProductName)
VALUES (101, 'Laptop');

INSERT INTO Orders
VALUES (102, 'Mouse', 'Delivered');
```

Result:

OrderID	ProductName	Status
101	Laptop	Pending
102	Mouse	Delivered

*Output: SELECT * FROM Orders; (Status defaulted to Pending for OrderID 101)*



Explanation: For OrderID 101, the Status column was omitted. MySQL automatically assigned Pending.

For OrderID 102, the value Delivered was explicitly provided, so the default was not used.

6. NOT NULL Constraint

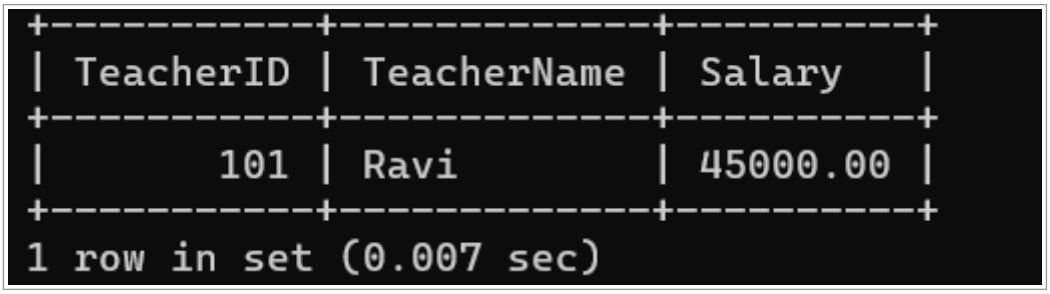
The NOT NULL constraint ensures that a column cannot contain NULL values.

```
CREATE TABLE Teachers (
    TeacherID INT PRIMARY KEY,
    TeacherName VARCHAR(50) NOT NULL,
    Salary DECIMAL(10,2)
);
```

Valid insertion:

```
INSERT INTO Teachers
VALUES (101, 'Ravi', 45000);
```

*Output: SELECT * FROM Teachers; (valid record stored)*

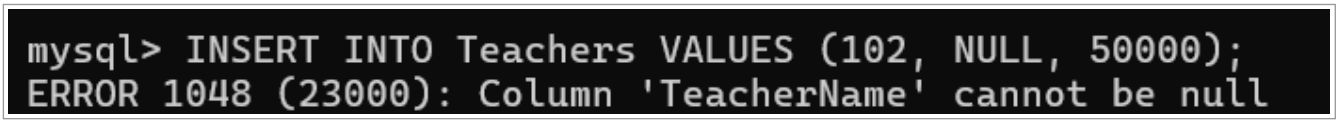


Invalid insertion:

```
INSERT INTO Teachers
VALUES (102, NULL, 50000);
```

Error: Column 'TeacherName' cannot be NULL.

Output: NOT NULL violation (TeacherName is NULL)



Adding Constraints Using ALTER TABLE

Constraints can also be added after creating a table.

Constraint	SQL command
PRIMARY KEY	ALTER TABLE Students ADD PRIMARY KEY (StudentID);
FOREIGN KEY	ALTER TABLE Students ADD CONSTRAINT fk_dept FOREIGN KEY (DeptID) REFERENCES Departments(DeptID);
UNIQUE	ALTER TABLE Students ADD UNIQUE (Email);
CHECK	ALTER TABLE Students ADD CONSTRAINT chk_age CHECK (Age >= 18);
DEFAULT	ALTER TABLE Students ALTER COLUMN City SET DEFAULT 'Chennai';
NOT NULL	ALTER TABLE Students MODIFY StudentName VARCHAR(50) NOT NULL;